

21.4 Lines, Line Segments, and Polygons

A *line* is defined by any two points through which it passes. Call them **a** and **b**. As in §21.1, the points can be two-dimensional, if the domain of interest is a plane, or three-dimensional (or higher), if the line is embedded in a higher-dimensional space. For now, consider only the two-dimensional case.

Parametrically, any point **c** that lies on the line defined by **a** and **b** must be a linear combination of those two points. One way to write this is

$$\mathbf{c} = \mathbf{a} + s(\mathbf{b} - \mathbf{a}) \quad (-\infty < s < \infty) \quad (21.4.1)$$

where s is a parameter along the line. The chosen normalization is to make $s = 0$ at **a** and $s = 1$ at **b**. The part of the line between **a** and **b** has $0 \leq s \leq 1$ and is a *line segment*, denoted $\overline{\mathbf{ab}}$. The whole line is denoted $\overleftrightarrow{\mathbf{ab}}$.

The easiest way to get the equation satisfied by all points **c** on the line $\overleftrightarrow{\mathbf{ab}}$ is to take the vector cross product of equation (21.4.1) with $(\mathbf{b} - \mathbf{a})$ on the right. Using the fact that the cross product of any vector with itself is zero, we get

$$\mathbf{c} \times (\mathbf{b} - \mathbf{a}) = \mathbf{a} \times \mathbf{b} \quad (21.4.2)$$

or writing out the components,

$$c_0(b_1 - a_1) - c_1(b_0 - a_0) = a_0b_1 - a_1b_0 \quad (21.4.3)$$

which is indeed a linear relation between the coordinates c_0 and c_1 . While it is tempting to divide this equation by $b_0 - a_0$ to get an equation in that old familiar high school form “ $y = mx + b$,” one should often resist that temptation since, as written, equation (21.4.3) remains valid for the case of a vertical line, when $b_0 - a_0 = 0$.

Intersection of Two Lines. In the plane, two lines $\overleftrightarrow{\mathbf{ab}}$ and $\overleftrightarrow{\mathbf{xy}}$ most always intersect. We can solve for the point of intersection by equating the two lines’ parametric forms,

$$\mathbf{a} + s(\mathbf{b} - \mathbf{a}) = \mathbf{x} + t(\mathbf{y} - \mathbf{x}) \quad (21.4.4)$$

and then solving the two equations (components 0 and 1) for the two unknowns s and t . The result is

$$\begin{aligned} s &= \frac{(\mathbf{x} - \mathbf{y}) \times (\mathbf{a} - \mathbf{x})}{(\mathbf{b} - \mathbf{a}) \times (\mathbf{x} - \mathbf{y})} = \frac{(x_0 - y_0)(a_1 - x_1) - (x_1 - y_1)(a_0 - x_0)}{(b_0 - a_0)(x_1 - y_1) - (b_1 - a_1)(x_0 - y_0)} \\ t &= \frac{(\mathbf{a} - \mathbf{x}) \times (\mathbf{b} - \mathbf{a})}{(\mathbf{b} - \mathbf{a}) \times (\mathbf{x} - \mathbf{y})} = \frac{(a_0 - x_0)(b_1 - a_1) - (a_1 - x_1)(b_0 - a_0)}{(b_0 - a_0)(x_1 - y_1) - (b_1 - a_1)(x_0 - y_0)} \end{aligned} \quad (21.4.5)$$

Of course, the special case of parallel lines with no intersection is indicated by the vanishing of the denominators.

All those cross products might make you think that equation (21.4.5) has a geometrical interpretation. Indeed so. In Figure 21.4.1, the lines intersect at **o**. Segment $\overline{\mathbf{x}\mathbf{o}}$ is therefore just $\overline{\mathbf{xy}}$ scaled by t , while $\overline{\mathbf{a}\mathbf{o}}$ is similarly $\overline{\mathbf{ab}}$ scaled by s . The area of $\triangle \mathbf{o}\mathbf{x}\mathbf{a}$ is therefore given (cf. equation 21.3.1) by

$$2\mathcal{A}(\mathbf{o}\mathbf{x}\mathbf{a}) = st (\mathbf{x} - \mathbf{y}) \times (\mathbf{a} - \mathbf{b}) \quad (21.4.6)$$

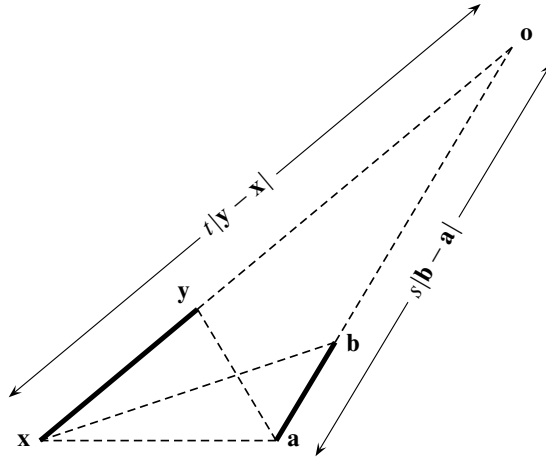


Figure 21.4.1. Geometrical construction that yields the intersection point of two lines in terms of ratios of triangle areas. See text for details.

By linearity of a triangle's area with its height (holding the base fixed), we also have

$$\mathcal{A}(\mathbf{o}\mathbf{x}\mathbf{a})/\mathcal{A}(\mathbf{y}\mathbf{x}\mathbf{a}) = t \quad \mathcal{A}(\mathbf{o}\mathbf{x}\mathbf{a})/\mathcal{A}(\mathbf{b}\mathbf{x}\mathbf{a}) = s \quad (21.4.7)$$

Equation (21.4.5) follows immediately from these relations and equation (21.3.1).

Point-to-Line Distance. What is the perpendicular distance d from an arbitrary point $\mathbf{q}$ to the line $\overleftrightarrow{\mathbf{ab}}$ that passes through points $\mathbf{a}$ and $\mathbf{b}$? Evidently d is the height of $\triangle \mathbf{abq}$ when its base is the segment $\mathbf{ab}$. Therefore, from the schoolbook “one-half base times height” formula,

$$d = \frac{2\mathcal{A}(\mathbf{abq})}{|\mathbf{a} - \mathbf{b}|} = \frac{(q_0 - b_0)(a_1 - b_1) - (q_1 - b_1)(a_0 - b_0)}{\sqrt{(a_0 - b_0)^2 + (a_1 - b_1)^2}} \quad (21.4.8)$$

Note that d is signed, positive if it is to the left of the directed line from $\mathbf{a}$ to $\mathbf{b}$, negative if it is to the right, and a good segue to our next topic.

21.4.1 Line Segment Intersections and “Left-Of” Relations

You can use equation (21.4.5) to test whether two line segments, $\overline{\mathbf{ab}}$ and $\overline{\mathbf{xy}}$, intersect: Calculate s and t and then check if they are both in the range $(0, 1)$. (To keep our discussion brief, we won't say much here, or in what follows, about the various degenerate cases where s or t or both are exactly 0 or 1. These are straightforward to figure out if your application so demands.)

A related, but slightly different, approach is to use the fact that the formulas for triangle areas given in equation (21.3.1) are signed. This means that, as equivalent statements, we have

$$\mathcal{A}(\mathbf{abc}) > 0 \iff \left[\begin{array}{l} \mathbf{c} \text{ is to the left of line } \overleftrightarrow{\mathbf{ab}} \\ \text{when it is traversed in} \\ \text{the direction from } \mathbf{a} \text{ to } \mathbf{b} \end{array} \right] \quad (21.4.9)$$

while $\mathcal{A}(\mathbf{abc}) < 0$ implies that $\mathbf{c}$ is to the right of the same line. We refer to either statement in (21.4.9) as a *left-of relation*.

A necessary and sufficient condition for two segments $\overline{ab}$ and $\overline{xy}$ to intersect is that x and y be on opposite sides of $\overleftrightarrow{ab}$, and a and b be on opposite sides of $\overleftrightarrow{xy}$. (We again omit discussion of the various special cases of collinearity.) This test, using the triangle area formulas in equation (21.3.1), involves evaluating four left-of relations, each computationally a cross product, which is just slightly more work than computing s and t (which share a denominator). However, you can sometimes use the same cross products, once computed, in other parts of your calculation. So, it is often a toss-up whether to use the “ s, t ” method or the “left-of” method — you should consider both.

Fig.	Δabx	Δaby	Δxya	Δxyb	Intersection	Hull
1	−	+	+	−	$\overline{ab} \times \overline{xy}$	$\square axby$
2	+	−	−	+	$\overline{ab} \times \overline{xy}$	$\square aybx$
3	+	−	−	−	$\overrightarrow{ab} \times \overline{xy}$	Δayx
4	−	+	−	−	$\overrightarrow{ba} \times \overline{xy}$	Δbyx
5	+	−	+	+	$\overrightarrow{ba} \times \overline{xy}$	Δbxy
6	−	+	+	+	$\overrightarrow{ab} \times \overline{xy}$	Δaxy
7	−	−	−	+	$\overrightarrow{yx} \times \overline{ab}$	Δyba
8	−	−	+	−	$\overrightarrow{xy} \times \overline{ab}$	Δxba
9	+	+	−	+	$\overrightarrow{xy} \times \overline{ab}$	Δxab
10	+	+	+	−	$\overrightarrow{yx} \times \overline{ab}$	Δyab
11	−	−	−	−	external	$\square ayxb$
12	+	+	−	−	external	$\square abxy$
13	+	+	+	+	external	$\square abxy$
14	−	−	+	+	external	$\square axyb$
	−	+	−	+	Not possible!	
	+	−	+	−	Not possible!	

Table 21.4.1. Relationship between two line segments classified by the signs of the areas of various triangles. Refer to Figure 21.4.2 for an illustration of each case.

Table 21.4.1 enumerates the 16 cases that you get if you compute all four possible “left-of” tests for two line segments. (Actually, only 14 are geometrically possible!) The table classifies each possibility as to whether the segments intersect (intersection denoted by $\times$, not to be confused with vector cross product!), whether the unidirectional extension of one segment (a *ray*) intersects the other segment, or (lastly) whether an ordinary line intersection occurs external to both segments. Also shown for each case is the outer *hull* of the two segments (the smallest triangle or quadrilateral enclosing both segments) and how it is traversed in clockwise order. Figure 21.4.2 shows an example of each possibility.

You can use the table to find combinations that test for specific circumstances.

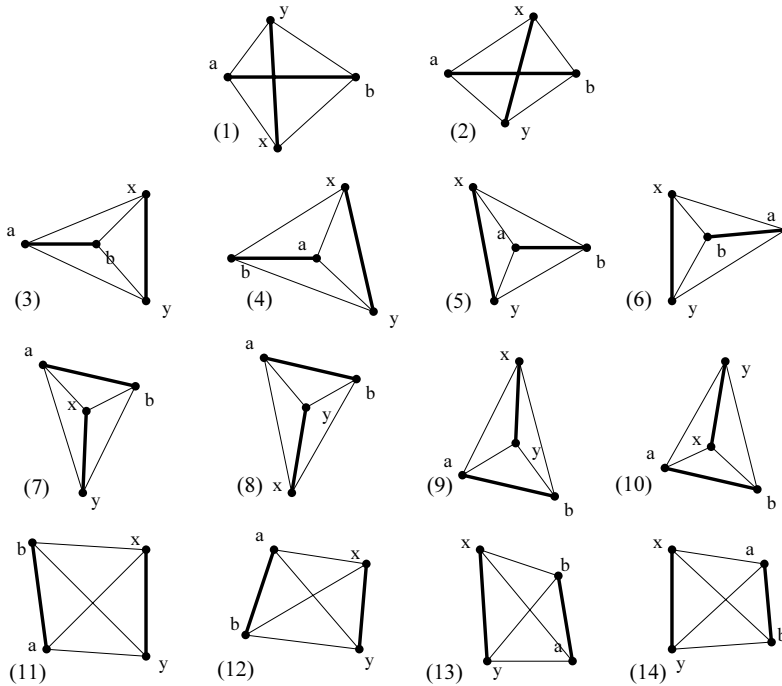


Figure 21.4.2. Two line segments, ab and xy , define four triangles ($\triangle abx$, $\triangle aby$, $\triangle xya$, and $\triangle xyb$), each of which can have positive or negative area. Of the 16 combinations of signs, 14 (shown here) correspond to to possible intersection relationships between the line segments or their extensions as rays.

For example, if you need a test for whether the ray $\overrightarrow{ab}$ intersects the segment $\overline{xy}$ (including the possibility of the segments intersecting), you examine rows 1, 2, 3, and 6 in the table and read off a test that involves just three left-of relations:

$$\mathcal{A}(\mathbf{abx})\mathcal{A}(\mathbf{aby}) < 0 \quad \text{and} \quad \mathcal{A}(\mathbf{aby})\mathcal{A}(\mathbf{xya}) > 0 \quad (21.4.10)$$

Of course there are exactly equivalent tests using s and t , for this example (with s and t as in equation 21.4.4) $s > 0$ and $0 < t < 1$.

Angle Between Two Vectors. Suppose that $\mathbf{U}$ and $\mathbf{V}$ are difference vectors along each of two lines, and that θ is the angle between the lines (measured from $\mathbf{U}$ to $\mathbf{V}$). In the previous notation, $\mathbf{U} = \mathbf{y} - \mathbf{x}$ and $\mathbf{V} = \mathbf{b} - \mathbf{a}$. Elementary vector analysis tells us that

$$\begin{aligned} \mathbf{U} \cdot \mathbf{V} &= U_0 V_0 + U_1 V_1 = |\mathbf{U}| |\mathbf{V}| \cos(\theta) \\ \mathbf{U} \times \mathbf{V} &= U_0 V_1 - U_1 V_0 = |\mathbf{U}| |\mathbf{V}| \sin(\theta) \end{aligned} \quad (21.4.11)$$

Many people try to get θ by using one or the other of the above relations, computing the vector norms and taking an inverse cosine or inverse sine. Big mistake! Not only are there quadrant ambiguities in the inverse trig functions, but there are also angles near the flat extrema of the sine and cosine functions where you can lose up to half of your significant figures in the answer. Not to mention the need to calculate square roots for the norms! The better approach is

$$\theta = \text{atan2}(\mathbf{U} \times \mathbf{V}, \mathbf{U} \cdot \mathbf{V}) = \text{atan2}(U_0 V_1 - U_1 V_0, U_0 V_0 + U_1 V_1) \quad (21.4.12)$$

where $\text{atan2}()$ is C's or C++'s quadrant-sensitive arctangent function. That function allows either of its arguments to be zero and returns a value in the range $-\pi/2$ to $\pi/2$. (An identical function exists in Fortran and most other languages.)

21.4.2 Lines in Three Dimensions

The immediate generalization of equation (21.4.12) to three-dimensional space gives the angle between two 3-vectors,

$$\theta = \text{atan2}(|\mathbf{U} \times \mathbf{V}|, \mathbf{U} \cdot \mathbf{V}) \quad (21.4.13)$$

Note the occurrence of the modulus of the vector cross product, which requires taking a square root.

Brevity constraints allow us to say only a little more about lines in three-dimensional space. The parameterization

$$\mathbf{c} = \mathbf{a} + s \mathbf{v} \quad (-\infty < s < \infty) \quad (21.4.14)$$

(equation 21.4.1 with $\mathbf{v} \equiv \mathbf{b} - \mathbf{a}$) still works, with $\mathbf{a}$, $\mathbf{v}$, and $\mathbf{c}$ now points in 3-space. The parameter s at which a line intersects a plane specified by $\mathbf{N}$ and D (see equation 21.3.16) is given by

$$s = \frac{D - \mathbf{a} \cdot \mathbf{N}}{\mathbf{v} \cdot \mathbf{N}} \quad (21.4.15)$$

with the denominator vanishing if the line is parallel to the plane.

The closest approach of a line to a point $\mathbf{q}$ occurs when

$$s = \frac{(\mathbf{q} - \mathbf{a}) \cdot \mathbf{v}}{|\mathbf{v}|^2} \quad (21.4.16)$$

You can also use this to see whether a line intersects a sphere in 3-space: Calculate the closest point on the line to the sphere's center, and then check if the distance is less than the sphere's radius (or compare squares of distances to avoid the square root).

Two lines, call them $\mathbf{a} + s \mathbf{v}$ and $\mathbf{x} + t \mathbf{u}$, will not, in general, share a common point; rather, they will be *skew* to one another. However, their points of closest approach can be calculated as [2]

$$s = \frac{\det\{(\mathbf{a} - \mathbf{x}), \mathbf{u}, \mathbf{u} \times \mathbf{v}\}}{|\mathbf{u} \times \mathbf{v}|^2} \quad t = \frac{\det\{(\mathbf{a} - \mathbf{x}), \mathbf{v}, \mathbf{u} \times \mathbf{v}\}}{|\mathbf{u} \times \mathbf{v}|^2} \quad (21.4.17)$$

where $\det$ is the 3×3 determinant whose columns are the indicated 3-vectors. The denominator vanishes if the lines are parallel. If you really must check for an actual intersection, plug these values for s and t into the parametric forms for each line, and check whether the distance between two points thus obtained is less than some roundoff tolerance.

A common operation in computer graphics is to test whether a line intersects a triangle in three dimensions. To do this with methods already discussed, use equation (21.3.17) to get $\mathbf{N}$ and D for the triangle's plane. Then use equations (21.4.14) and (21.4.15) to get the line's intersection with that plane. Finally, use equation (21.3.19) to get the barycentric coordinates α and β of the intersection. If α , β , and $\gamma \equiv 1 - \alpha - \beta$ are all positive, then the intersection is inside the triangle. See [4,1] for various ways to streamline this procedure.

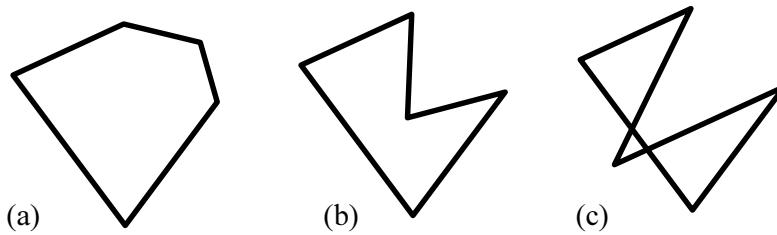


Figure 21.4.3. Polygons are classified as simple if they have no intersecting edge, as in (a) and (b), or complex (c) otherwise. Simple polygons are either convex (a) or concave (b).

21.4.3 Polygons

We define a *polygon* as a vector of N points (*vertices*), numbered from 0 to $N - 1$, and the N directed line segments that connect them in cyclic order, that is 0 to 1, 1 to 2, $\dots$, $N - 2$ to $N - 1$, and (importantly!) $N - 1$ to 0. (In some formulas below we will use the convention that vertex N is to be taken as meaning vertex 0.)

We consider two polygons to be the same if they differ only by a cyclical renumbering of the points, so that all their line segments are the same. However, if we reverse the order of traversing the points, we consider the resulting polygon to be different. (For example, the sign of its area will change.) If the boundary of a region cannot be traversed by a single cyclical vector (e.g., the region between an outer square and an enclosed triangle), we don't call it a polygon; other conventions are of course also possible.

With the definition given, it is useful to classify a polygon as either *simple*, meaning that none of its N line segments intersect, or *complex* if there are one or more segment intersections. We classify simple polygons according to whether they are convex or concave. A convex polygon can be defined either by (i) all $N(N - 1)/2$ line segments connecting two vertices lie in its interior (or on its boundary), or (ii) its exterior angles all have the same sign (zeros allowed). Whichever property is taken as the definition, the other becomes a theorem. Figure 21.4.3 shows examples of the three types.

For simple polygons, the sum of the exterior angles is always $\pm 2\pi$. That is, you turn through exactly one circle in driving around the polygon. If the polygon is concave, then the sign of the exterior angles must be taken into account when doing the sum. This is shown in Figure 21.4.4. The sign of the 2π is positive for a counterclockwise (CCW) traversal, negative for clockwise (CW).

Complex (that is, self-intersecting) polygons can also have exterior angles that sum to 2π , as the polygon in Figure 21.4.3(c), so the exterior angles do not provide, in general, any magical way of finding intersections. However, one small bit of magic does exist: If the exterior angles of a polygon are all of one sign (or zero), and if they sum to $\pm 2\pi$, then the polygon is both simple and convex. This provides a very rapid way to test for the simple-and-convex case, but it does not distinguish between simple-concave and complex polygons. Doing so requires a detailed check for intersecting edges (which we will implement in code below).

Winding Number. If you sit on a point $\mathbf{p}$ in the plane, and watch someone drive around a polygon, then they will drive around *you* some net integer number of times (with the usual sign convention, CCW being positive). This is the polygon's *winding number with respect to a point*. For simple polygons, the winding number is 1 for

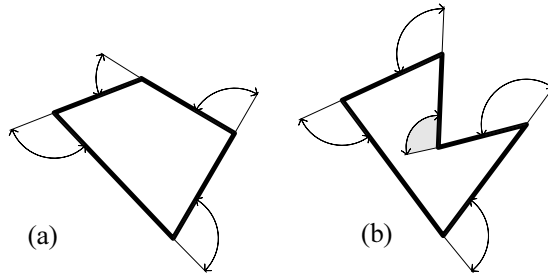


Figure 21.4.4. The exterior angles of simple polygons sum to one full circle. (a) If the polygon is convex, all the angles have the same sign. (b) If the polygon is concave, one or more angles (here, the shaded angle) has opposite sign.

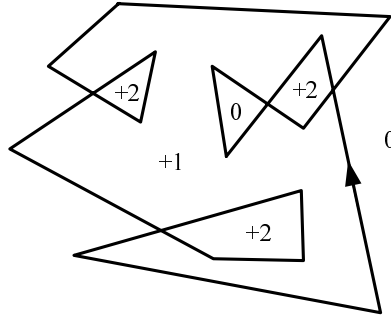


Figure 21.4.5. Complex polygon with different winding numbers (indicated by integers) around points in different regions. The total winding number of the polygon (sum of exterior angles divided by 2π) is 3, a value not realized in any single region. Note that interior regions can have winding number 0.

points inside a CCW polygon, -1 for points inside a CW polygon, and 0 for points outside. For complex polygons, however, there are no simple rules. Figure 21.4.5 shows a complicated case. Note that interior regions of a complex polygon can have winding number 0 , so a point's winding number (alone) does not determine whether it is inside or outside a complex polygon. Note also that the sum of a polygon's exterior angles, divided by 2π , is not necessarily the winding number of that polygon with respect to any point in the plane.

Doubtless the *worst* way to compute a polygon's winding number with respect to a point $\mathbf{q}$ is to add up the N incremental angles between $\mathbf{q}$ and consecutive vertices $\mathbf{p}_i$ of the polygon, that is,

$$\text{W.N.}(\mathbf{q}) = \frac{1}{2\pi} \sum_{i=0}^{N-1} \angle(\mathbf{p}_{i+1}\mathbf{q}\mathbf{p}_i) \quad (21.4.18)$$

(with the usual convention $\mathbf{p}_N \equiv \mathbf{p}_0$). Even using the trick in equation (21.4.12) to get the angles, there is an enormous amount of unnecessary computation in this approach.

Instead, we can observe that if a polygon winds M times around $\mathbf{q}$, then its edges must cross any ray from $\mathbf{q}$ to infinity a net of exactly M times, where ray crossings in the CCW direction are counted as positive, CW as negative. In particular, if we take the ray to be the horizontal ray to the right of $\mathbf{q}$, we can immediately

reject edges that fail to cross the horizontal line that contains $\mathbf{q}$, and then check for the ray crossing (and its sign) with a single “left-of” test [5]. These ideas are embodied in the following routine.

polygon.h

```
Int polywind(const vector< Point<2> > &vt, const Point<2> &pt) {
    Return the winding number of a polygon (specified by a vector of vertex points vt) around an
    arbitrary point pt.
    Int i,np, wind = 0;
    Doub d0,d1,p0,p1,pt0,pt1;
    np = vt.size();
    pt0 = pt.x[0];
    pt1 = pt.x[1];
    p0 = vt[np-1].x[0];
    p1 = vt[np-1].x[1];
    for (i=0; i<np; i++) {
        d0 = vt[i].x[0];
        d1 = vt[i].x[1];
        if (p1 <= pt1) {
            if (d1 > pt1 &&
                (p0-pt0)*(d1-pt1)-(p1-pt1)*(d0-pt0) > 0) wind++;
        }
        else {
            if (d1 <= pt1 &&
                (p0-pt0)*(d1-pt1)-(p1-pt1)*(d0-pt0) < 0) wind--;
        }
        p0=d0;
        p1=d1;
    }
    return wind;
}
```

Save last vertex as "previous" to first.

Loop over edges.

Upward-crossing edge. Is pt to its left?

Downward-crossing edge. Is pt to its right?

Current vertex becomes previous one.

Is there a similarly efficient way to find the total winding number of a polygon $\mathbf{p}_i$ ($i = 0, \dots, N-1$), that is, the sum of its exterior angles divided by 2π ? Yes. Consider the derived polygon whose vertex points $\mathbf{q}_i$ are given by the vector differences

$$\mathbf{q}_i = \mathbf{p}_{i+1} - \mathbf{p}_i \quad (i = 0, \dots, N-1) \quad (21.4.19)$$

Then the winding number of this derived polygon *around the origin* is just the total winding number of the original polygon. (Draw a picture if this isn't immediately obvious to you.) The routine `polywind()` can be used for the computation.

Point Inside Polygon. How can you tell whether an arbitrary point $\mathbf{q}$ is inside or outside a polygon [5]? Let us first assume that your polygon is known to be simple. For simple polygons, two commonly used approaches are the “winding number method” and the “Jordan curve theorem method.” However, when these are each implemented efficiently, they become virtually identical!

The winding number method is simply to compute the winding number of the polygon around the point (e.g., using `polywind()`, above). If the answer is ± 1 , then the point is inside the polygon. If it is zero, it is outside. Any other answer indicates that the polygon wasn't simple as assumed.

The Jordan curve theorem method observes that any ray from the point to infinity will cross the polygon an odd number of times if the point is inside, or an even number of times if it is outside [Figure 21.4.6(a)]. If we implemented this in code, it would be almost identical to the code in `polywind` except for one detail: Instead of incrementing or decrementing a counter at each ray crossing (according to the direction of the crossing), we would always increment it. Then, at the end, we would

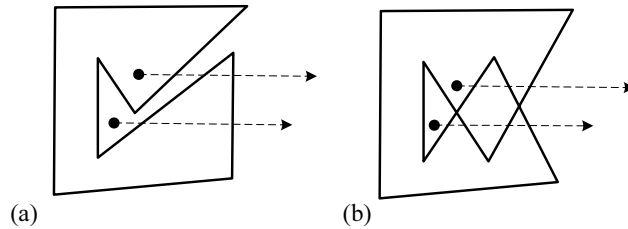


Figure 21.4.6. Is a point inside a polygon? (a) For a simple polygon, either the winding number, or the Jordan curve theorem (even or odd number of crossings of a ray) can be used. (b) For complex polygons, there is no simple test.

check whether the counter is even or odd. But if `polywind` as written returns 0, it *must* have encountered the same number of increments as decrements, hence an even number of crossings. And if it returns ± 1 (the only other possible value for a simple polygon), it must similarly have encountered an odd number. So the two methods are really the same.

What if your polygon is not simple? As Figure 21.4.6(b) illustrates, you are in deep waters. Both the winding number method and the Jordan curve theorem method will say that the upper point in the figure is inside the complex polygon shown, and this seems intuitively correct. However, both methods will say that the lower point is *outside* the polygon. Indeed, there are some self-consistent ways of defining “insideness” for complex polygons that make this the case. The result is so counterintuitive, however, as to be useless in most practical applications. It is generally better just to avoid using the idea of “insideness” with complex polygons.

Classification of Polygons. We are now in a position to combine several of the ideas already introduced into a function that classifies any polynomial as either simple or complex, and (if it is simple) whether it is convex or concave, and whether it is CCW (total winding number 1) or CW (total winding number -1).

```
Int ispolysimple(const vector< Point<2> > &vt) {
```

[polygon.h](#)

Classifies a polygon specified by a vector of vertex points `vt`. Returns 0 if the polygon is complex (has intersecting edges). Returns ± 1 if it is simple and convex. Returns ± 2 if it is simple and concave. The sign of the returned value indicates whether the polygon is CCW (+) or CW (−).

```
    Int i, ii, j, jj, np, schg=0, wind=0;           Initialize sign change and winding number.
    Doub p0, p1, d0, d1, pp0, pp1, dd0, dd1, t, tp, t1, t2, crs, crsp=0.0;
    np = vt.size();
    p0 = vt[0].x[0]-vt[np-1].x[0];
    p1 = vt[0].x[1]-vt[np-1].x[1];
    for (i=0, ii=1; i<np; i++, ii++) {           Loop over edges.
        if (ii == np) ii = 0;
        d0 = vt[ii].x[0]-vt[i].x[0];
        d1 = vt[ii].x[1]-vt[i].x[1];
        crs = p0*d1-p1*d0;                       Cross product at this vertex.
        if (crs*crsp < 0) schg = 1;              Sign change (i.e., concavity) found.
        if (p1 <= 0.0) {                          Winding number logic as in polywind.
            if (d1 > 0.0 && crs > 0.0) wind++;
        } else {
            if (d1 <= 0.0 && crs < 0.0) wind--;
        }
        p0=d0;
        p1=d1;
        if (crs != 0.0) crsp = crs;              Save previous cross product only if it has a
    }                                             sign!
```

```

if (abs(wind) != 1) return 0;           Can already conclude polygon is complex.
if (schg == 0) return (wind>0? 1 : -1); Polygon is simple and convex.
Drat, we've exhausted all the quick tricks and now have to check all pairs of edges for
intersections:
for (i=0,ii=1; i<np; i++,ii++) {
    if (ii == np) ii=0;
    d0 = vt[ii].x[0];
    d1 = vt[ii].x[1];
    p0 = vt[i].x[0];
    p1 = vt[i].x[1];
    tp = 0.0;
    for (j=i+1,jj=i+2; j<np; j++,jj++) {
        if (jj == np) {if (i==0) break; jj=0;}
        dd0 = vt[jj].x[0];
        dd1 = vt[jj].x[1];
        t = (dd0-d0)*(p1-d1) - (dd1-d1)*(p0-d0);
        if (t*tp <= 0.0 && j>i+1) {      First loop is only to compute starting tp,
            pp0 = vt[j].x[0];           hence test on j.
            pp1 = vt[j].x[1];
            t1 = (p0-dd0)*(pp1-dd1) - (p1-dd1)*(pp0-dd0);
            t2 = (d0-dd0)*(pp1-dd1) - (d1-dd1)*(pp0-dd0);
            if (t1*t2 <= 0.0) return 0;  Found an intersection, so done.
        }
        tp = t;
    }
}
return (wind>0? 2 : -2);               No intersections found, so simple concave.
}

```

When `ispolysimple` finds that the quick indicators are not enough, and that it needs to check all pairs of edges for intersections, it does so by the obvious $O(N^2)$ method of two nested loops. For small N , say less than 10, this is likely as fast as any other strategy. If you are dealing with large numbers of large- N polygons, however, you will want to substitute a method with better scaling in N . One way, using the code from §21.8, would be to define a class for segments with a `collides()` method, then store the segments into a QO tree one at a time, looking for collisions at each step. (Don't forget that adjacent edges of a simple polygon are allowed to "collide" at their shared vertex.)

Area of Polygons. As a next topic, let us turn to the *area* of a polygon. The (signed) area of a polygon is the sum of the areas of each of its regions weighted by that region's winding number. For simple polygons the area is thus what you would expect geometrically, except that its sign will be negative for a polygon traversed CW rather than CCW. (We previously saw this in the special case of triangles.) For a complex polygon like that shown in Figure 21.4.5, the answer is less intuitive (and generally less useful) since some regions, such as the interior region with winding number 0, are not counted at all, while others are counted (in this case) twice.

The great advantage of this definition of area, however, is that it results in a simple expression for the area that applies to both simple and complex polygons. Let x_i and y_i be, respectively, the 0 and 1 coordinates of the polygon's vertex $\mathbf{p}_i$, and let $\mathcal{A}$ be the polygon's area. Then, in three equivalent forms,

$$2 \mathcal{A} = \sum_{i=0}^{N-1} x_i y_{i+1} - x_{i+1} y_i$$

$$\begin{aligned}
&= \sum_{i=0}^{N-1} (x_{i+1} + x_i)(y_{i+1} - y_i) \\
&= \sum_{i=0}^{N-1} x_i (y_{i+1} - y_{i-1})
\end{aligned} \tag{21.4.20}$$

Evaluation of any of these forms takes just one loop over the polygon's vertices. (These formulas go back as far as Meister in 1769 and Gauss in 1795.)

Although we won't derive equation (21.4.20) in detail, the middle form does have an intuitive interpretation. It sums the areas of trapezoids each with two points on the y -axis ($x = 0$) at y_i and y_{i+1} , and with the other two points the points on the polygon at these y 's. In going around the polygon, negative-area trapezoids subtract from positive-area ones so as to leave just the area inside.

Interestingly, there are very similar formulas for the x and y coordinates of the centroid or center-of-mass of an arbitrary polygon [3],

$$\begin{aligned}
\bar{x} &= \frac{1}{6} \sum_{i=0}^{N-1} (x_{i+1} + x_i)(x_i y_{i+1} - x_{i+1} y_i) \\
\bar{y} &= \frac{1}{6} \sum_{i=0}^{N-1} (y_{i+1} + y_i)(x_i y_{i+1} - x_{i+1} y_i)
\end{aligned} \tag{21.4.21}$$

Note the common subexpressions with equation (21.4.20), so that it is efficient to calculate the area and centroid position together.

Finally, a couple of polygon tidbits for your edification or amusement:

- If two simple polygons have the same area, then the first can be cut into a finite number of polygonal pieces that can be reassembled into the second. This is known as the Bolyai-Gerwien theorem. (The corresponding statement about polyhedra in three dimensions, "Hilbert's Third Problem," was proved false by Dehn in 1900.)
- The regular polygon with N sides is constructible with a compass and straight-edge if the factorization of N contains only the factors 2, 3, 5, 17, 257, 257, and 65537 (whose odd members are the *Fermat primes*), with each odd factor occurring at most once. It is not known whether any other N -gons are also constructible; but, if so, then their N must contain a factor at least as large as $2^{2^{33}} + 1$. The product of the known Fermat primes, which is perforce the largest known constructible polygon with an odd number of sides, is $2^{32} - 1 = 4294967295$, a number well known to computer trolls as the largest positive 32-bit integer. Go figure.

CITED REFERENCES AND FURTHER READING:

- Bowyer, A. and Woodwark, J. 1983, *A Programmer's Geometry* (London: Butterworths).
- Schneider, P.J. and Eberly, D.H. 2003, *Geometric Tools for Computer Graphics* (San Francisco: Morgan Kaufmann), §11.1.2[1]
- de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 2.